Année 2025-2026

Mentor : Mr Arnol Namegni

Projet 13

Passez un systeme IA du POC au MVP

Exercice sur l'entreprise Puls-Events

Candidat : Mr Jean-Francois Chaussoy

Alternant Data Engineering

Puls-Events - Plateforme web de découverte d'événements culturels en temps réel

- **Source :**
  OpenAgenda + autres API

- **Personnalisation :**
  lieu, période, thématiques

- **Objectif stratégique :**
  positionner Puls-Events à l'avant-garde de la transformation numérique dans l'événementiel



Rappel du POC (Projet 11)

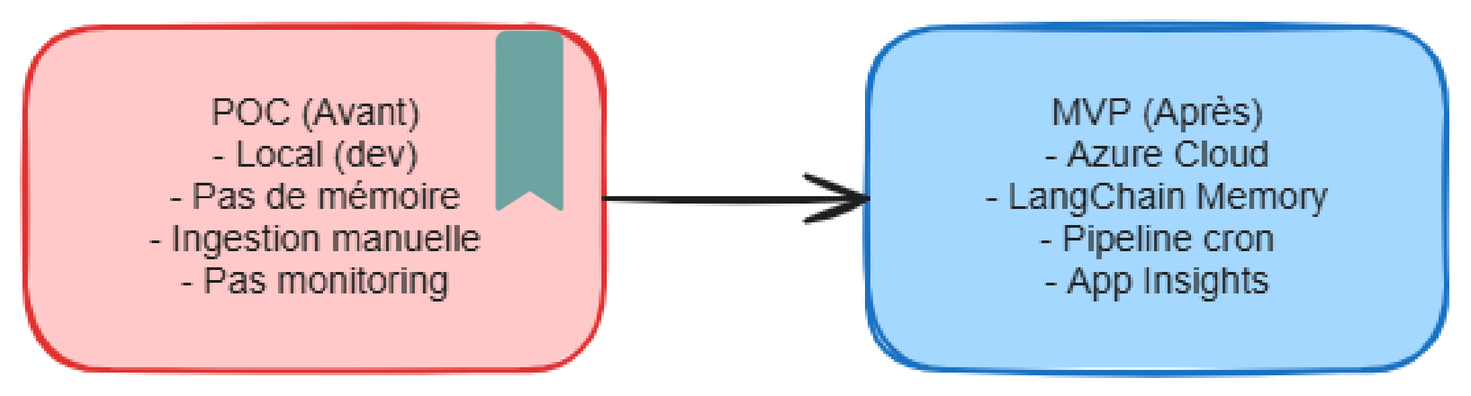
Le POC confirme la faisabilité d'un chatbot RAG sur le corpus testé.

<u>Éléments réalisés :</u>

- 485 événements après nettoyage et déduplication ;
- 495 vecteurs FAISS de dimension 1 024 ;
- évaluation exploratoire : 89,6 % de similarité cosinus, sur 5 questions ;
- index et métadonnées sauvegardés sur disque local.

Limites du POC :

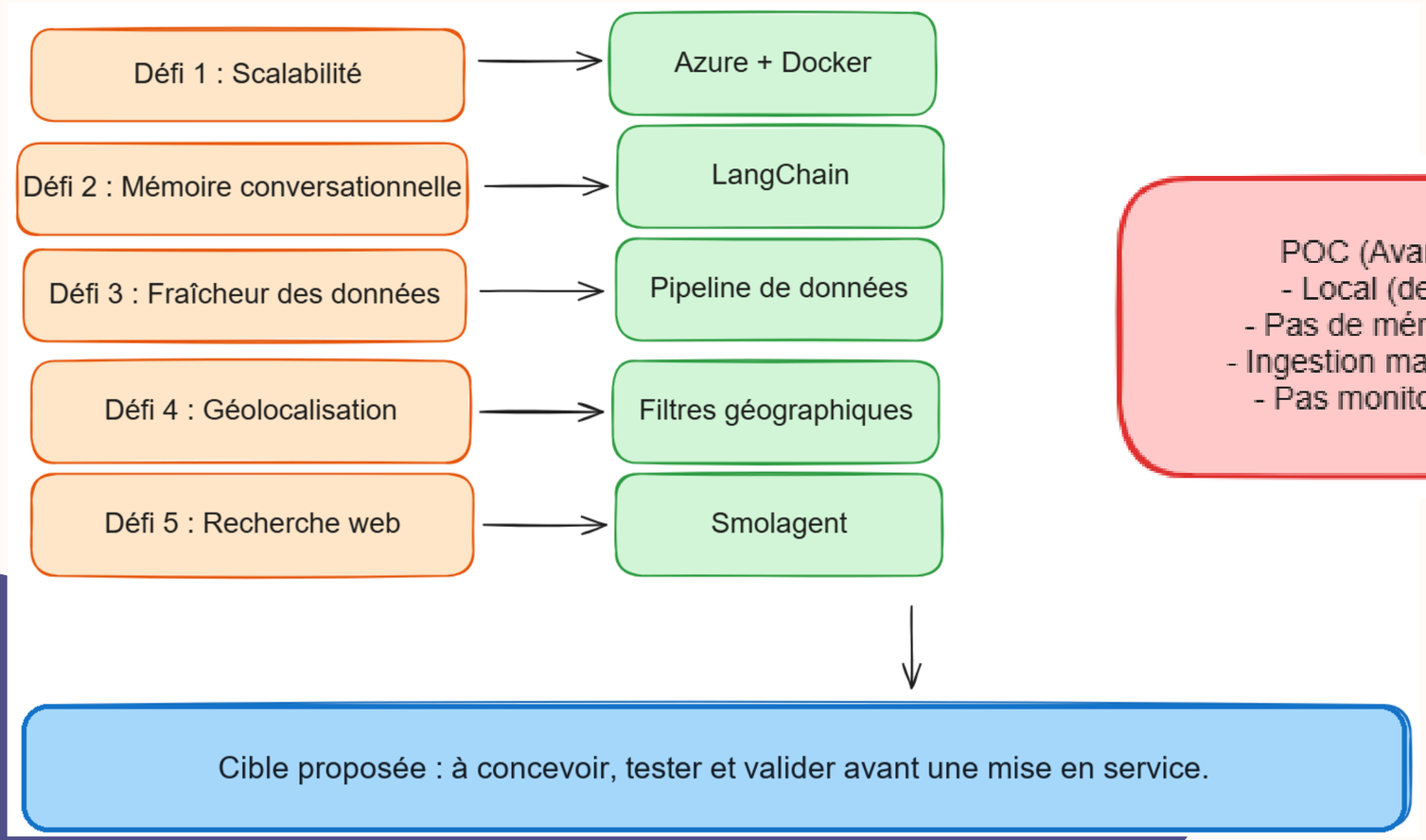
- console utilisable question par question ; aucune API multi-utilisateur ;
- l'historique n'est pas transmis au moteur RAG ;
- ni ingestion planifiée, ni suivi de latence, d'erreurs API ou de coût.

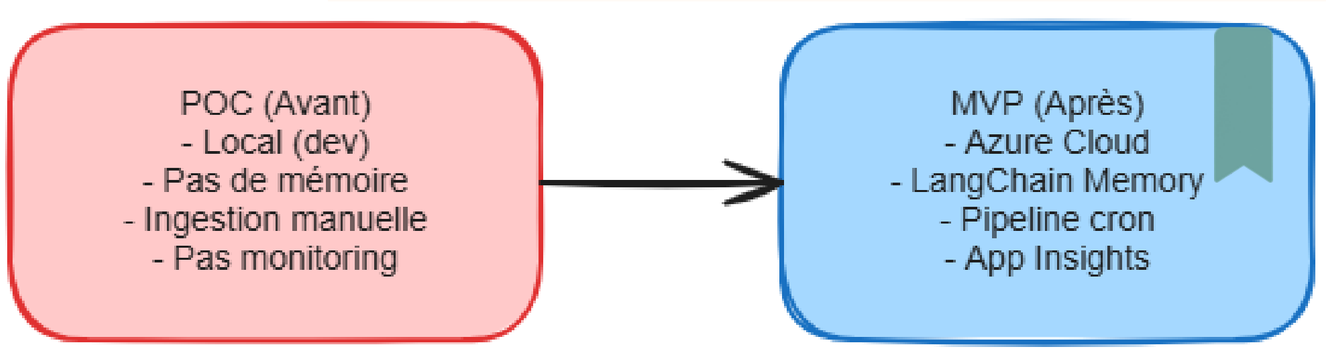


Pistes de travail pour le MVP

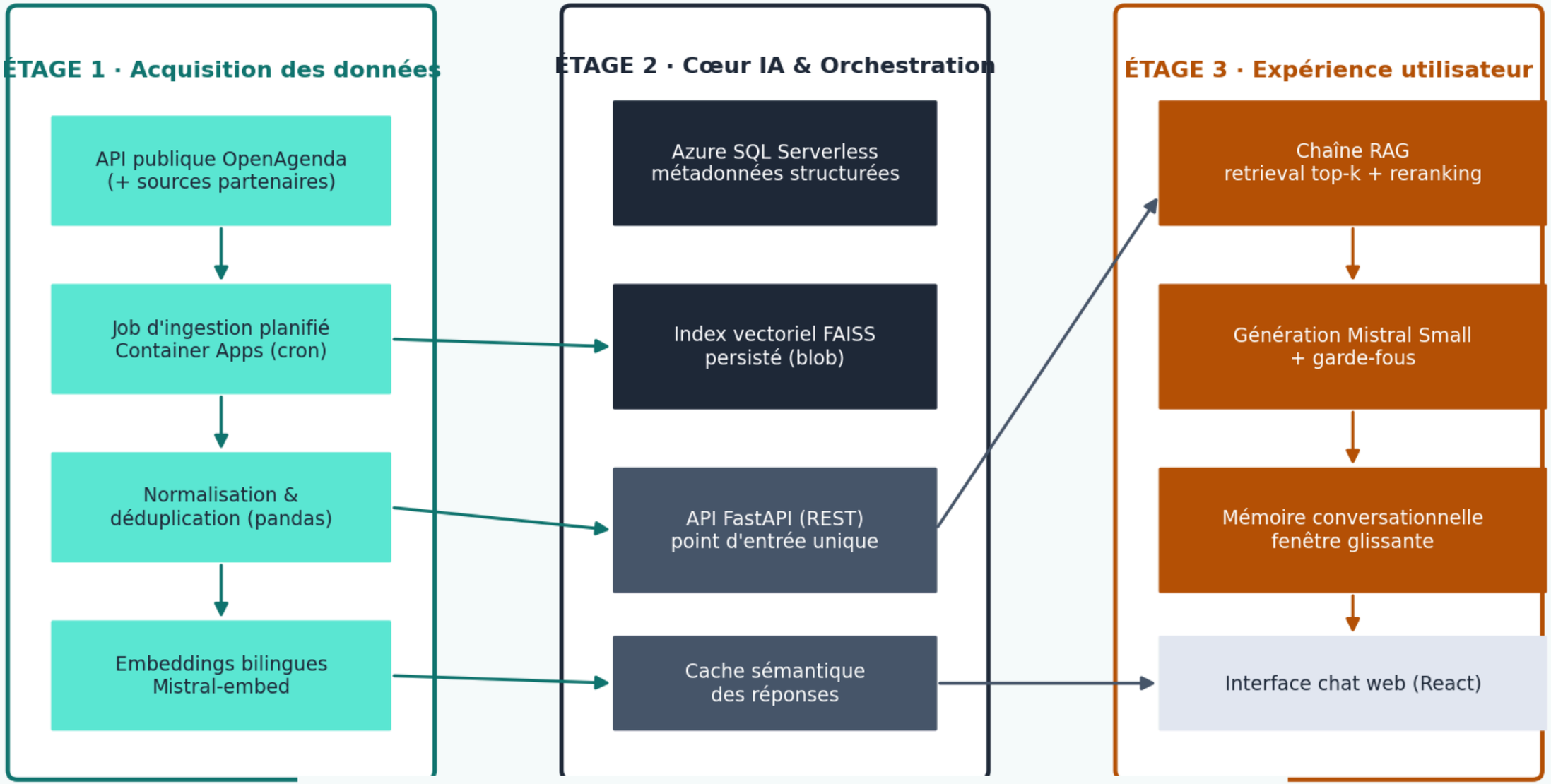
Pistes à prioriser et valider avant tout développement du MVP :

1. Mémoire de conversation : transmettre le contexte utile au moteur RAG.
2. Hébergement : comparer les options cloud et conteneuriser le service.
3. Fraîcheur : planifier puis contrôler le rafraîchissement du catalogue.
4. Personnalisation : tester l'usage du lieu dans les recommandations.
5. Recherche web : évaluer l'intérêt et le coût d'un enrichissement externe.
6. Suivi : mesurer qualité, latence, erreurs et retours utilisateurs.





Architecture cible proposée (3 couches)



Architecture cible proposée — à valider, non réalisée dans le POC

Options techniques envisagées — à valider

Brique	Technologie	Justification
Langage	Python 3.11	Écosystème IA/NLP documenté ; bibliothèques utilisées dans le POC et compétences déjà mobilisées.
API	FastAPI	API Python ; validation des données avec Pydantic et documentation OpenAPI générée. Charge et concurrence à tester.
RAG	LangChain + FAISS + API Mistral	LangChain orchestre le flux ; FAISS assure la recherche vectorielle locale ; génération via l'API Mistral. Persistance et partage de l'index à valider.
Hébergement	Azure Container Apps (Consumption)	Plateforme managée, sans gérer directement un cluster Kubernetes. Scale-to-zero possible avec minReplicas=0 et règles adaptées ; ressources facturées à la seconde. Tester le cold start.
Base	Azure SQL Database (Serverless, General Purpose)	Auto-pause configurable (60 min par défaut) ; compute facturé à la seconde et nul pendant la pause ; stockage toujours facturé. Tester la reprise.
Conteneurs	Docker	Reproductibilité entre développement, CI et déploiement. Docker ne supprime pas la dépendance aux services Azure retenus.
Monitoring	Azure Monitor, Log Analytics + Application Insights	Centraliser logs, métriques et traces. Instrumenter latence P95, erreurs API et tokens par requête ; chiffrer coûts et rétention.

Macro backlog

Scénarios cloud à comparer selon les besoins et le budget :

9 Must (26 jours) :

- ingestion planifiée, normalisation et déduplication ;
- persistance des données et de l’index ;
- API, chaîne RAG avec sources et tests de qualité ;
- conteneurisation, intégration continue et recette.

2 Should (3 jours) :

- **mémoire de conversation et suivi applicatif.**

2 Could (2 jours) :

- filtre géographique et recherche web, si validés.

13 fonctionnalités — estimation sur 31 jours

Estimation des coûts — hypothèses à valider

BUILD — estimation initiale

31 jours-hommes × 460 €/j

14 260 € HT, hors aléas et frais externes

OPEX — hypothèse (5 000 questions/mois)

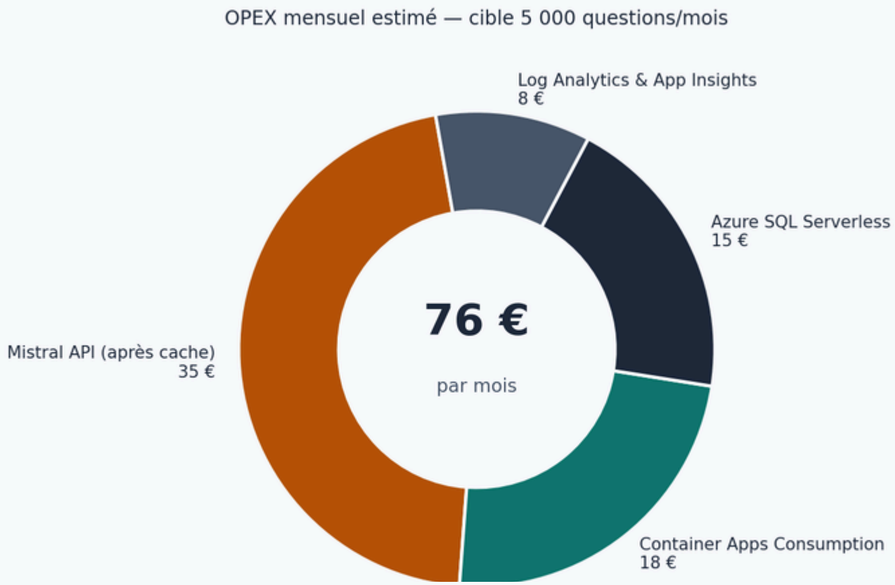
Mistral API : 35 €/mois, selon volume estimé

Container Apps : 18 € ; Azure SQL : 15 € ; supervision : 8 €

Total indicatif : environ 76 €/mois

Tarifs, consommation de tokens et taux de cache à confirmer par un test.

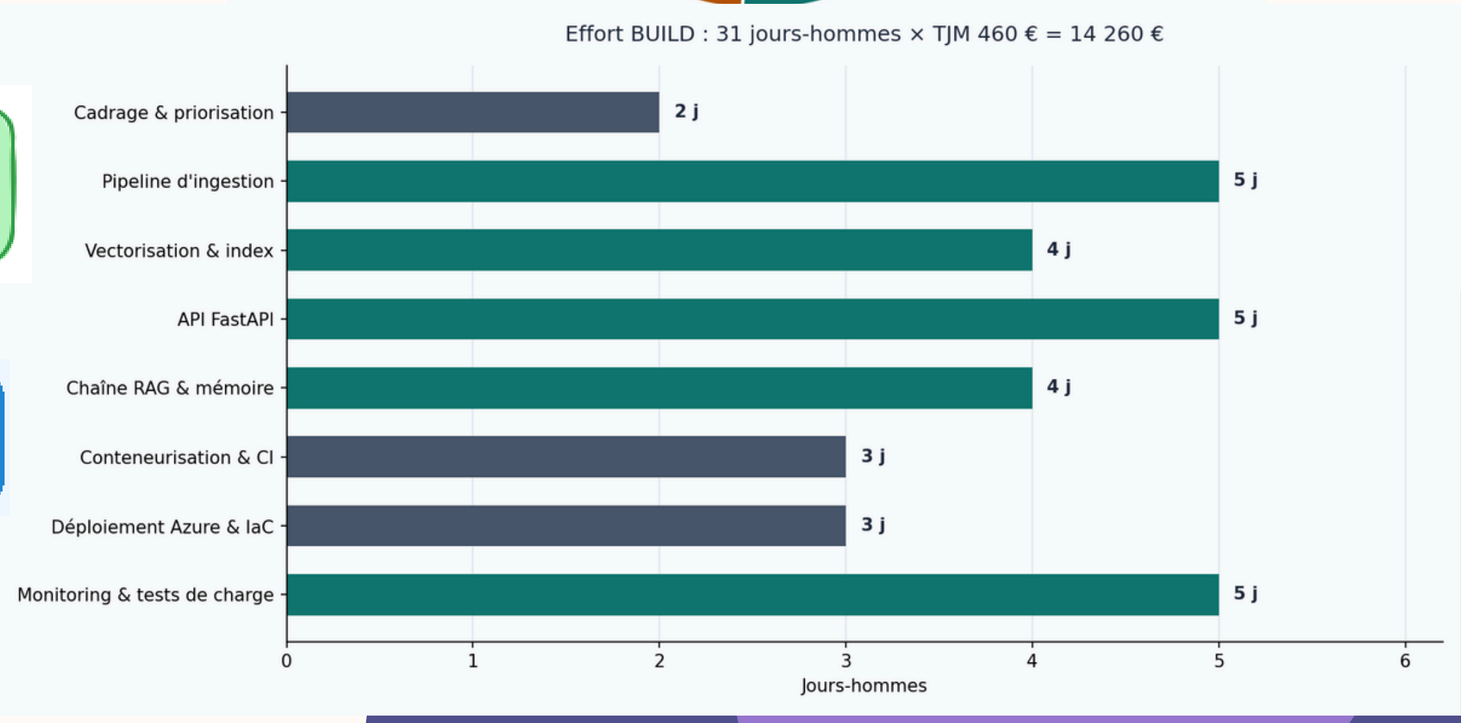
Aucun de ces montants n’a été mesuré en production.



BUILD (one-shot)
31j × 460€ = 14 260€ HT
8 lots, profil DE junior

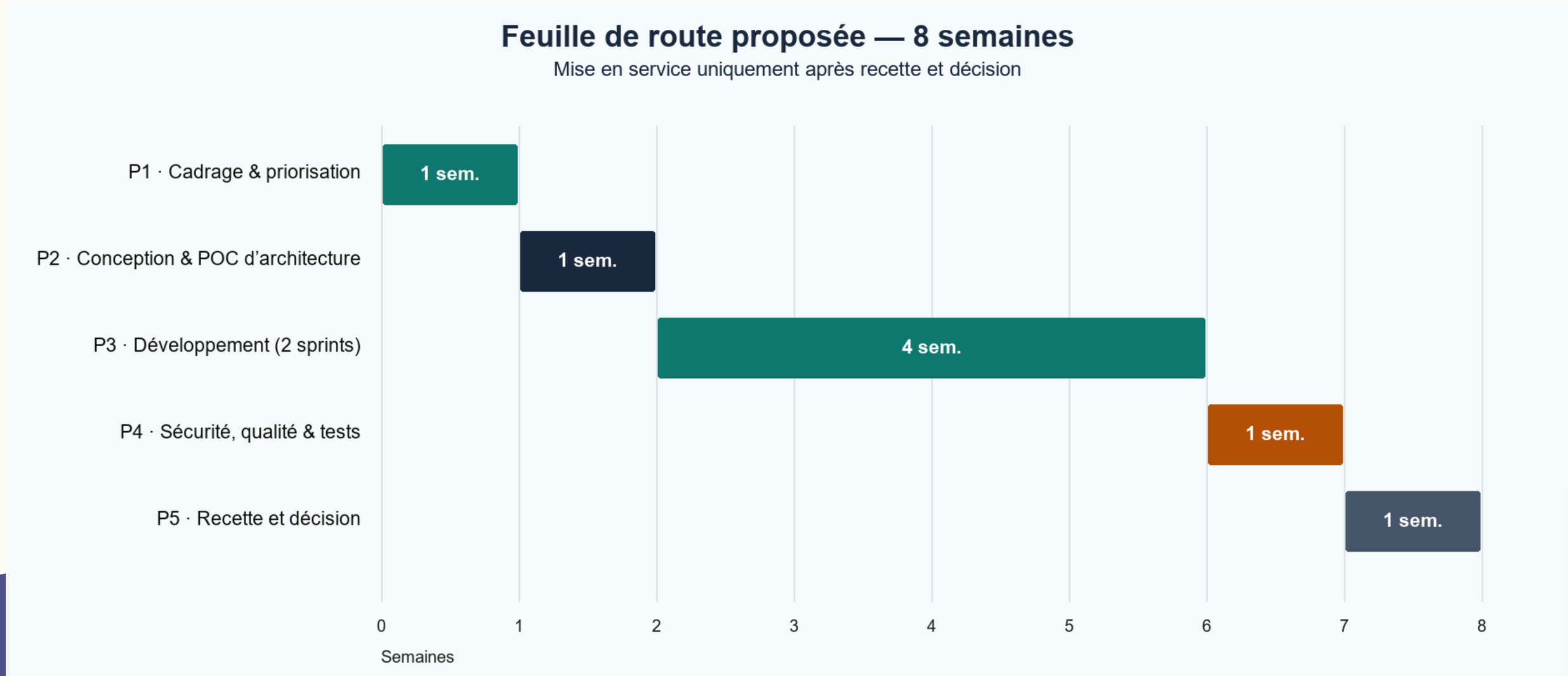
OPEX (~76€/mois)
Mistral 35 + Container 18
+ SQL 15 + Monitor 8

Hypothèse de calcul : 5 000 questions par mois
Coûts et taux de cache à confirmer par un test.



Plan de projet — calendrier indicatif et jalons

S8 : recette, puis décision de mise en service



Gestion des risques & Méthodologie

Démarche proposée : cadrage puis itérations courtes.
Cadrage : préciser le périmètre, les options et les hypothèses.
Réalisation : itérations de deux semaines, à confirmer avec l'équipe.
Validation : tester qualité, sécurité et charge avant décision.
Suivi prévu : latence P95, erreurs API, coût par requête et retours utilisateurs.

Les seuils seront définis après les premiers tests.

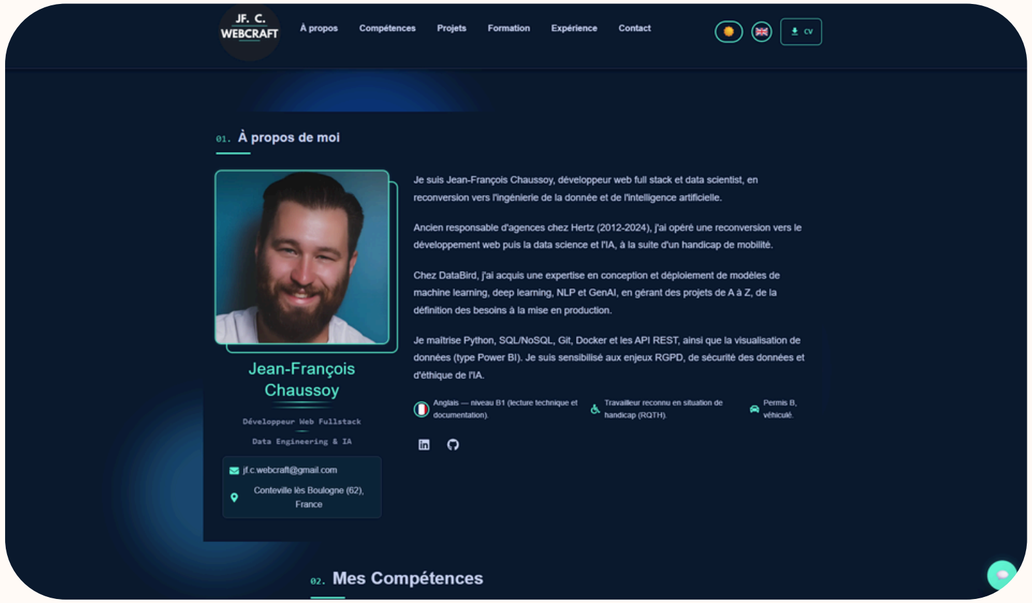
Risques à suivre :

Quotas LLM → mesurer les appels et prévoir une solution de repli.
Réponses inexactes → tester des cas connus et afficher les sources.
Dépassement budgétaire → recalculer les coûts et définir des alertes.
Données périmées → définir et contrôler la fréquence d'ingestion.
Les pistes ci-dessous ne sont pas encore déployées.

- Quotas API → mesurer les appels et prévoir un repli
- Réponses inexactes → tester et afficher les sources
- Budget → suivre les dépenses, définir des alertes
- Dépendance fournisseur → évaluer une alternative
- Données → valider les champs et les doublons

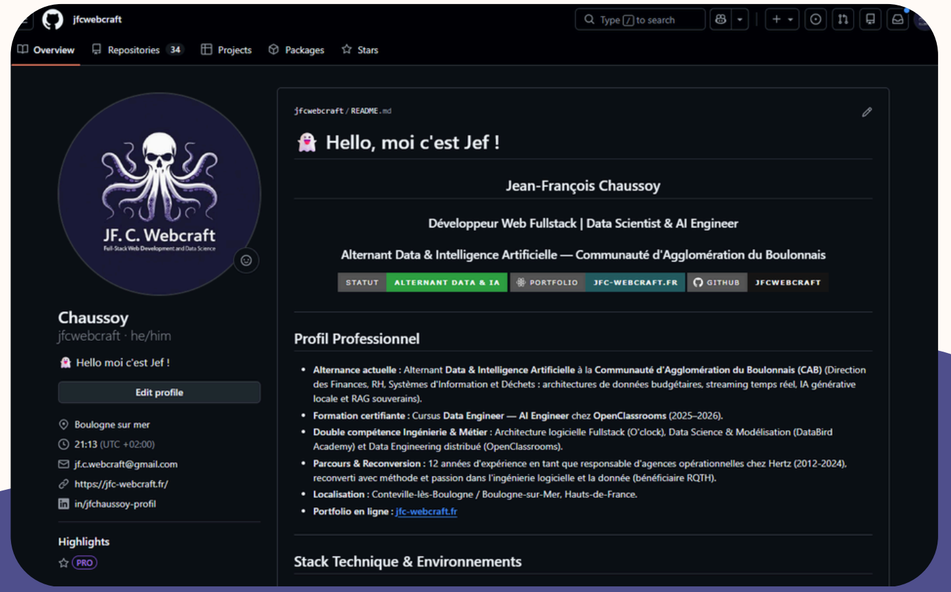
Portfolio Professionnel & Conclusion

Mon Portfolio en ligne
- Site Web : https://jfc-webcraft.fr/



Portfolio GitHub : github.com/jfcwebcraft/JF.C.-Webcraft-Portefolio

- Projet GitHub : github.com/jfcwebcraft/Projet-13-Passez-votre-systeme-IA-du-POC-au-MVP



Conclusion :
Le POC est réalisé. Le MVP, lui, reste à concevoir, chiffrer et valider.

Merci pour votre attention !

